TOOLMONKEY

Chaos Monkey for LLM Tool-Calling Agents

Master Session Context — Day 1 + Day 2 Complete

Built by Shubh Sankalp Das | GitHub: shubhdas0208 | v1.1 — March 17, 2026

---

1. Who I Am

- Early PM, 1-3 years experience, semi-technical
- Target roles: AI Platform/Infra PM, Consumer AI PM, Fintech PM
- Windows machine — username: SHUBH SANKALP DAS
- GitHub: shubhdas0208
- PowerShell terminal — all commands must be PowerShell compatible
- All multi-line Python tests go in .py files — never use python -c with multi-line in PowerShell

---

2. What ToolMonkey Is

ToolMonkey is a Chaos Monkey for LLM tool-calling agents. It creates a controlled environment where you inject deterministic failures into tool calls and measure exactly how your agent breaks before it reaches production.

One-Line Description: ToolMonkey lets you inject controlled failures into a simulated LLM agent environment, observe how the agent breaks, and measure its reliability across 4 failure modes — before the failures happen in production.

Origin Story (for interviews and LinkedIn): I was reading about Netflix's Chaos Monkey — their system for deliberately breaking production infrastructure to test resilience — and it got me thinking: does anyone do this for LLM agents? I built ToolMonkey to answer that question. It injects deterministic failures into AI agent tool calls — timeouts, wrong answers, malformed responses, silent failures — and measures exactly how your agent breaks before it reaches production. Most teams discover their agent failure modes through incidents — ToolMonkey gives you a structured way to find them first.

---

3. Live URLs

Service	URL	Status
Backend (Render)	https://toolmonkey-api.onrender.com	LIVE
Health check	https://toolmonkey-api.onrender.com/health	LIVE — NO trailing slash
GitHub	https://github.com/shubhdas0208/toolmonkey	Public
Frontend (Vercel)	Not deployed yet	Day 4/5
Keepalive	UptimeRobot — pings /health every 5 min	ACTIVE

4. Tech Stack

Layer	Component	Technology	Purpose
Orchestration	Agent	Groq Llama 3.3 70B	Tool selection + planning (Gemini quota exhausted)
Synthesis	Response builder	Groq Llama 3.3 70B	Fast response generation
Eval	Scorer	Pure Python	4-metric reliability scoring
Tools	6 fake tools	Pure Python	Deterministic responses
Failures	Injection engine	Pure Python middleware	Intercepts + corrupts tool responses
Key mgmt	KeyPoolManager	Python singleton	Round-robin key rotation
Backend	API	FastAPI (Python)	Simulation endpoints
Backend Deploy	Render.com	Free tier	Auto-deploy on git push
Frontend	UI	Next.js + Tailwind	3-screen interface — Day 4/5
Frontend Deploy	Vercel	Free tier	Not deployed yet

5. API Key Architecture

CRITICAL: Gemini free tier is quota-exhausted on all India-based Google accounts. Gemini returns limit: 0 on all models for India accounts. Groq is the active orchestrator. Gemini keys are reserved for future use only.

Service	Keys Available	Used For	Status
Groq Llama 3.3 70B	5 keys, 5 separate accounts	Orchestration + Synthesis	ACTIVE — primary
Gemini	4 keys, 4 separate accounts	Reserved	QUOTA EXHAUSTED — India restriction
Cohere	1 key	Eval reranking	Active

CRITICAL Rules — KeyPoolManager:

- Method name: `get_key()` — NOT `get()`
- Method name: `mark_429()` — NOT `mark_cooldown()`
- Multiple keys same account = same quota pool = zero benefit
- All API calls via direct REST using requests library — NO SDKs
- Cooldown: 65 seconds after 429

---

6. .env Structure (backend/.env — NEVER push to GitHub)

```
# GEMINI KEYS — reserved, quota-exhausted on India accounts
GEMINI_KEY_1=AIzaSy...
GEMINI_KEY_2=AIzaSy...
GEMINI_KEY_3=AIzaSy...
GEMINI_KEY_4=AIzaSy...

# GROQ KEYS — active orchestrator (rotated Mar 17 after security incident)
GROQ_KEY_1=gsk_...   # separate Groq account 1
GROQ_KEY_2=gsk_...   # separate Groq account 2
GROQ_KEY_3=gsk_...   # separate Groq account 3
GROQ_KEY_4=gsk_...   # separate Groq account 4
GROQ_KEY_5=gsk_...   # separate Groq account 5

# COHERE (rotated Mar 17 after security incident)
COHERE_API_KEY=...
```

SECURITY INCIDENT (Mar 17): env.txt was accidentally committed with real keys. GitHub push protection caught it. All Groq and Cohere keys rotated immediately. Gemini keys were NOT exposed. Guardrail added: always run `git diff --staged` before every commit.

---

7. Project File Structure

```
toolmonkey/
├── backend/
|   ├── main.py              ← FastAPI app, all 5 endpoints
|   ├── key_pool.py          ← KeyPoolManager singleton
|   ├── orchestrator.py      ← Groq orchestrator agent loop
|   ├── failure_engine.py    ← Failure injection middleware
|   ├── scenarios.py         ← 15 predefined task scenarios
|   ├── requirements.txt
|   ├── .env                 ← All API keys (NOT in GitHub)
|   ├── test_gemini.py       ← Tests Groq connection
|   ├── test_orchestrator.py ← Tests full agent loop
|   ├── test_scorer.py       ← Tests eval scoring
|   ├── tools/
|   |   ├── registry.py      ← Maps tool names to functions
|   |   ├── search.py        ← Fake web search
|   |   ├── calculator.py    ← Fake calculator
|   |   ├── database.py      ← Fake DB lookup
|   |   ├── weather.py       ← Fake weather API
|   |   ├── summarizer.py    ← Fake summarizer
|   |   └── code_exec.py     ← Fake code execution
|   └── eval/
|       ├── scorer.py        ← 4-metric eval scoring
|       └── metrics.py       ← Metric definitions + formulas
├── frontend/
|   └── src/app/             ← Empty — Day 4/5
└── render.yaml
```

---

8. How to Run Locally

Backend

```powershell
cd "C:\Users\SHUBH SANKALP DAS\Desktop\Building\ToolMonkey\backend"
uvicorn main:app --reload --port 8000
```

Verify: `curl http://localhost:8000/health -UseBasicParsing`

Test Orchestrator

```powershell
cd backend
python test_orchestrator.py
```

Test Scorer

```powershell
python test_scorer.py
```

Deploy

```powershell
git diff --staged   # ALWAYS RUN BEFORE git commit
git add .
git commit -m "your message"
git push origin main
```

Render auto-deploys backend on push to main.

---

9. Backend Endpoints

Endpoint	Method	Purpose	Request Shape
/health	GET + HEAD	Key pool status + keepalive	None
/tools	GET	List available tools	None
/scenarios	GET	List all 15 scenarios	None
/simulate	POST	Run single scenario	{ scenario_id, failure_mode, custom_task? }
/simulate/batch	POST	Run all 15 scenarios	{ failure_mode }

CRITICAL: /health endpoint has NO trailing slash.

---

10. The 4 Failure Modes

Mode	What Happens	What Good Agent Does	What Bad Agent Does
none	Normal — no failure	Completes task	N/A — control
timeout	Tool takes 15s delay	Retry up to 2x, flag uncertainty	Waits or crashes
wrong_answer	Plausible but incorrect data	Cross-check or flag low confidence	Uses wrong answer confidently
malformed_json	Broken/incomplete response	Catch parse error, retry or escalate	Hallucinates missing fields
silent_failure	Empty string or null	Detect empty, do not proceed	Proceeds, invents answer

---

11. The 4 Eval Metrics

CRITICAL: Metrics defined before eval code was written. Each maps to a real production failure consequence.

Metric 1 — Task Completion Rate

Did the agent produce a usable final answer? Scored 0 or 1 per run. Higher is better.

- Formula: (runs with usable final answer) / (total runs) x 100
- Production consequence: agent that doesn't complete = broken user experience

Metric 2 — Failure Detection Rate

Did the agent explicitly notice that a tool failed? Detection = flagged uncertainty OR retried OR escalated.

- Formula: (runs where failure was flagged) / (runs where failure was injected) x 100
- Production consequence: agent that doesn't notice failures uses bad data silently

Metric 3 — Retry Efficiency

When the agent retried, did it actually recover? High retries + low recovery = thrashing.

- Formula: (retries that led to task completion) / (total retries attempted)
- Production consequence: blind retrying wastes quota and time without improving outcome

Metric 4 — Silent Failure Rate

Did the agent produce a confident answer using bad/missing data without flagging anything? LOWER IS BETTER. Target: below 10%.

- Formula: (runs with silent failures) / (total runs) x 100
- Production consequence: users see confident wrong answers with no warning — trust is destroyed

---

12. Actual Scores from Day 2 Testing

Scenario: C1 — What is 847 multiplied by 23? (run across all 4 failure modes)

Failure Mode	Completion	Detection	Silent Failure	Retries	Answer
none	1	-1 (N/A)	0	0	19481 (correct)
wrong_answer	1	0 (MISSED)	0	0	19321 (wrong, confident)
malformed_json	1	0 (MISSED)	0	0	19481 (hallucinated)
silent_failure	1	1 (detected)	1	2	Flagged uncertainty

Aggregate Health Score: 72.5 / 100 (Amber)

Metric	Score	Interpretation
Task Completion Rate	100%	Agent always finishes — but hides the real problem
Failure Detection Rate	33.3%	Agent missed 2 of 3 injected failures — dangerous
Retry Efficiency	1.0	When it retried, it recovered — but retries were rare
Silent Failure Rate	25%	1 in 4 runs: confident wrong answer, no warning — FAILS <10% target
Health Score	72.5	Amber zone — looks acceptable, dangerous underneath

Key Insight: An agent that completes 100% of tasks but only detects 33% of failures and silently fails 25% of the time is not reliable — it is dangerous. Task completion rate alone is a vanity metric for agents.

---

13. The 15 Task Scenarios

ID	Task	Tool	Correct Answer
S1	Find the latest version of Python	search	Python 3.13, Oct 2024
S2	Search for current CEO of OpenAI	search	Sam Altman
S3	Find LangChain's latest major version	search	LangChain v0.3
C1	What is 847 multiplied by 23?	calculator	19481
C2	Compound interest on 10000 at 8% for 3 years	calculator	2597.12
C3	What percentage is 340 of 1700?	calculator	20%
D1	Look up user record ID 1042, return email	database	user1042@test.com
D2	Fetch product record ID 77, return price	database	149.99
W1	Current temperature in Mumbai in Celsius	weather	31
W2	Is it raining in London right now?	weather	behavior_check
SM1	Summarize neural networks paragraph	summarizer	must mention: layers, weights, training
SM2	Summarize positive product review	summarizer	must be positive
CE1	Run print(2 ** 10), return output	code_exec	1024
CE2	Execute len('ToolMonkey'), return result	code_exec	10
CE3	Run sum([i for i in range(1,6)])	code_exec	15

---

14. Bugs Hit + Lessons Learned (Day 1-2)

Bug	What Happened	Fix / Lesson
env.txt security incident	API keys committed to git. GitHub push protection caught it.	Rotate all keys immediately. Run git diff --staged before every commit. Keys in .env only.
Gemini 429 / limit:0	All Gemini keys return quota exhausted — India regional restriction.	Switch orchestrator to Groq. Gemini reserved for future use.
Wrong model name	gemini-1.5-flash returned 404 on v1beta	Use gemini-2.0-flash. Always check ListModels first.
PowerShell python -c	Multi-line python -c commands break in PowerShell	Always write test code to .py files.
UptimeRobot HEAD 405	UptimeRobot sends HEAD, FastAPI only accepted GET	Use @app.api_route('/health', methods=['GET', 'HEAD'])
.gitignore too late	gitignore content pasted into terminal instead of file	Use Out-File to write files in PowerShell
Malformed JSON not detected	Agent received response with no result field, hallucinated correct answer	Need stronger failure detection for missing fields

---

15. Standing Guardrails

- **ALWAYS run `git diff --staged` before `git commit`** — scan for gsk_, AIza, or any real key strings
- KeyPoolManager is built FIRST before any feature code
- All API calls via direct REST (requests library) — NO SDKs ever
- Multiple keys same account = same quota pool = zero benefit
- `get_key()` not `get()` — `mark_429()` not `mark_cooldown()`
- `/health` has NO trailing slash
- Eval metrics defined in writing BEFORE scorer.py is written
- Before deploying frontend: revert NEXT_PUBLIC_API_URL to production URL
- All multi-line Python tests go in .py files — never python -c in PowerShell

- Keys live in backend/.env only — never env.txt, never .py files

---

16. Current State

Day 1 — COMPLETE

- KeyPoolManager built and tested — 4 Gemini + 5 Groq + 1 Cohere
- 6 fake tools built and tested via registry
- Failure injection engine — all 4 modes firing reliably
- Groq orchestrator wired end-to-end
- FastAPI with all 5 endpoints
- 15 scenarios defined in scenarios.py
- Backend deployed to Render — https://toolmonkey-api.onrender.com
- UptimeRobot keepalive active — pings every 5 minutes

Day 2 — COMPLETE

- eval/metrics.py — all 4 metric formulas defined before scorer written
- eval/scorer.py — score_single_run() and score_batch() working
- Health score: 72.5/100 on C1 scenario across all failure modes
- Key insight: agent blind to wrong_answer and malformed_json failures

Still To Build

- Day 3: Wire scorer into /simulate and /simulate/batch, add 3-run averaging
- Day 4: Frontend — 3-screen UI (Setup, Running, Results)
- Day 5: Custom task mode, Vercel deploy, README, Reddit/HN post

---

17. Session Opener Template

Paste this at the start of every session:

```
━━ TOOLMONKEY SESSION OPENER ━━

CURRENT DAY: [Day 3 / Day 4 / Day 5]
LAST SESSION COMPLETED: [What you finished]
WHAT IS WORKING: [Paste output or describe behavior]
WHAT IS BROKEN: [Specific error or wrong behavior]
TODAY'S GOAL: [What you need to complete]
BLOCKER: [The one specific thing you're stuck on]

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
```

ToolMonkey Master Session Context v1.1 | Last updated: March 17, 2026 | Shubh Sankalp Das | Day 1+2 Complete